JAVA

미림여자정보과학고등학교
뉴미디어 소프트웨어과
1106 김나은

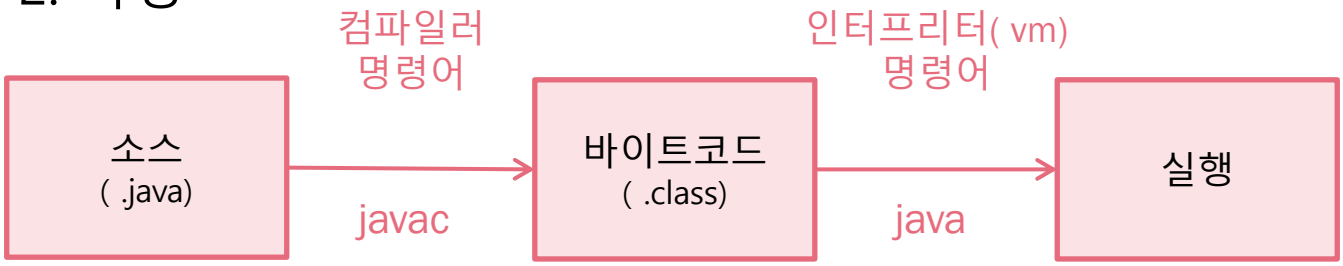
목차

1. JAVA 언어?
2. 특징
3. 자바
4. 변수 (변수, 데이터형)
5. 배열
6. 이차원 배열
7. 연산자
8. 제어문
9. 객체
10. 객체지향 프로그래밍
11. 객체지향 언어의 특징
12. 클래스
13. 생성자
14. 메소드와 오버로딩
15. 상속
16. 추상클래스
17. 인터페이스

1. JAVA 언어?

- 프로그래밍 언어 : 문제 해결 방법을 나타내기 위해 특정 단어와 기호를 사용한 명령문들의 집합
- 배경
  - 특정 컴퓨터마다 실행 파일을 만드는 C언어의 문제점
  - 가전 제품의 긴 수명으로 인한 완벽한 호환성 및 네트워크를 통한 시스템 업그레이드가 가능한 언어의 필요성
  - C++ 언어의 복잡성
- JAVA는...
  - 1991 선마이크로시스템즈의 제임스 고슬링을 주축으로 개발
  - 1991 Oak 언어 개발 : 가전 제품과 정보기기를 통합하는 새로운 인터페이스를 제공하는 언어 (현재 JAVA언어의 전 모습)
  - 1995 JAVA로 이름을 바꾸며 공개
  - 1996 JAVA 1.0 버전 발표

2. 특징



- ✓ 완벽한 객체지향언어
- ✓ 플랫폼에 독립적 (VM위에서 실행)
- ✓ 컴파일러와 인터프리터 언어 두 가지의 특징을 모두 가진다.
- ✓ 단순하고 안전하다.
- ✓ 분산 처리에 용이하다. (네트워크에 강함)
- ✓ 안전하다. (보안에 강함)
- ✓ 멀티스레드를 지원한다. (속도가 빠르다)
- ✓ 동적이다. (라이브러리의 확장이 용이하다)
- ✓ 견고하다

  포인터를 사용하지 않는다.

  실행시간에 발생하는 에러를 처리한다.

  자동으로 garbage collection 수행 → 사용하지 않는 객체는 자동으로 사라짐

  엄격한 데이터형 검사를 통해 에러를 조기에 발견

3. 자바

- 프로그래밍 언어 : 문제 해결 방법을 나타내기 위해 특정 단어와 기호를 사용한 명령문들의 집합

- 배경
  - 특정 컴퓨터마다 실행 파일을 만드는 C언어의 문제점
  - 가전 제품의 긴 수명으로 인한 완벽한 호환성 및 네트워크를 통한 시스템 업그레이드가 가능한 언어의 필요성
  - C++ 언어의 복잡성

- JAVA는...
  - 1991 선마이크로시스템즈의 제임스 고슬링을 주축으로 개발
  - 1991 Oak 언어 개발 : 가전 제품과 정보기기를 통합하는 새로운 인터페이스를 제공하는 언어 (현재 JAVA언어의 전 모습)
  - 1995 JAVA로 이름을 바꾸며 공개
  - 1996 JAVA 1.0 버전 발표

- 자바 개발환경 (JDK : Java Development Kit)

자바 프로그램
JAVA API : 자바가 지원하는 클래스 라이브러리 JVM (자바 가상 머신) : 바이트 코드를 다양한 플랫폼에 맞는 기계어로 해석하여 실행
하드웨어 의존적 플랫폼 (윈도우, 유닉스, 매킨토시)

종류	특징
J2SE	- 일반 자바 애플레케이션을 개발하기 위한 기본 자바 개발 환경
J2EE	- 서버 관련 프로그램을 개발하기 위한 자바 개발 환경 - 웹 서버 환경에서 많이 활용됨 (JSP, Servlet, EJB 등)
J2ME	- 다양한 제품과 임베디드 기기 프로그램을 개발하기 위한 자바 개발 환경 - 모바일 환경에서 많이 활용됨 (PDA, 핸드폰 등)

```
public class 클래스 이름 {
        public static void main(String args[ ]){
            system.out.println("Hello JAVA Wolrd");
            /*주
              석*/
            //주석
        }
}
class 클래스 이름{
        실행할 명령;
}
```

4. 변수 - 변수

- 변수란?

  데이터가 저장된 기억공간의 이름, 데이터는 필요에 따라 변할 수 있으므로 변수라고 함.

  ※ 변수의 작성 규칙

  - 알파벳 대.소문자, 숫자, _ , $ 만 사용할 수 있다.
  - 첫 문자로 숫자는 올 수 없다.
  - 자바 예약어는 사용할 수 없다.
  - 대.소문자를 구분한다.
  - 길이 제한은 없다.

4. 변수 - 데이터형

기본형

종류	데이터형	바이트 수	범위
논리형	boolean	1	true or false
단일 문자형	char	2	'lu0000' ~ 'luffff'
정수형	byte	1	-2^{7} ~ $2^{7}-1$
	short	2	-2^{15} ~ $2^{15}-1$
	int	4	-2^{31} ~ $2^{31}-1$
	long	8	2^{63} ~ $2^{63}-1$
실수형	float	4	1.4e-45 ~3.4028235e38
	double	8	4.9e -324 ~ 1.7976931348623157e308

5. 배열

※ 배열이란? ※

- 같은 종류의 데이터를 여러 개 저장하기 위한 기억 장소
- 자바는 배열을 객체로 처리. (참조형 변수)
- 메모리 절약, 쉽고 간결한 프로그램 작성 가능

※ 배열 사용 ※

1단계 : 배열 선언

기억 장소의 주소를 가리키는 변수를 선언. (배열의 크기 지정X)

ex) int a[ ]; , int [ ] a;

2단계 : 배열 생성

기억 공간을 할당하는 new 연산자로 배열 객체가 저장될 기억 공간을 확보하고

이 기억 공간의 주소를 1단계의 배열 변수에 저장. (배열의 크기를 지정)

** 배열객체 각 원소의 기억 공간을 접근하는 인덱스는 0부터 시작한다.

ex) a = new int[3];

3단계 : 배열 초기화

배열객체 각 원소의 기억 공간에 데이터를 초기화 한다.

ex) a[0] = 10; a[1] = 20; a[2] = 30;

1, 2 단계 한꺼번에 표현

ex) int a[ ] = new int[3]; 또는 int[ ] a = new int[3];

1, 2, 3 단계 한꺼번에 표현

ex) int [ ]a ={1, 2. 3}; 또는 int [ ] a = new int [ ]{1, 2, 3};

6. 이차원 배열

1단계 – int a[ ][ ]; int [ ] a [ ]; int [ ][ ]a;

2단계 – a = new int [2][3];

3단계 – a[0][0]=1; a[0][1]=2; a[0][2]=3; a[1][0]=10; a[1][1]=20; a[1][2]=30;

1	2	3
10	20	30

1, 2 단계

ex) int [ ][ ] a = new int[2][3];

1, 2, 3 단계

ex) int [ ][ ] a= {{1, 2, 3},{10, 20, 30}}

7. 연산자

산술 연산자

종류	사용법	기능	사용 예	
			연산식	결과
+	a + b	덧셈	20 + 10	30
-	a- b	뺄셈	20 – 10	10
*	a * b	곱셈	20 * 10	200
/	a / b	나눗셈	20 / 10	2
%	a % b	나머지	20 % 10	0

관계 연산자

종류	사용법	기능	사용 예	
			연산식	결과
>	a > b	크다	20 > 10	True
>=	a >= b	크거나 같다	20 >= 10	True
<	a < b	작다	20 < 10	False
<=	a <= b	작거나 같다	20 <= 10	False
==	a == b	같다	20 == 10	False
!=	a != b	같지 않다	20 != 10	True

논리 연산자

종류	사용법	기능	사용 예	
			연산식	결과
&	a & b	a와 b가 모두 true인 경우에만 true, 나머지 경우는 모두 false	(1>2) & (1<2)	F
&&	a && b	a와 b가 모두 true인 경우에만 true, 단, a가 false 인 경우, b를 수행하지 않고 false.	(1>2) && (1<2)	F
|	a | b	a와 b 중 하나라도 true면, true, a와 b 모두 false면, false.	(1>2) | (1<2)	T
||	a || b	a와 b 중 하나라도 true면 true 반환, 단, a가 true인 경우, b를 수행하지 않고 true.	(1>2) || (1<2)	T
!	!a	a가 true면, flase, false면 ture.	!(1<2)	F

증감 연산자

종류	사용법	기능	사용 예	
			연산식	결과
++	++a	먼저 변수 a 값 1 증가 후, 수식 적용.	a = 10; b = ++a + 10;	a = 11 b = 21
	a++	수식 적용 후, 나중에 변수 a 값 1 증가.	a = 10; b= a++ + 10;	b = 20 a = 11
- -	- - a	먼저 변수 a 값 1 감소 후, 수식 적용.	a = 10; b= - -a + 10;	a = 9 b = 19
	a - -	수식 적용 후, 나중에 변수 a 값 1 감소.	a = 10; b= a- - + 10;	b = 20 a = 9

비트 연산자

종류	사용법	기능	사용 예	
			연산식	결과
&	a & b	비트 별로 두 비트가 모두 1이면 1, 하나라도 0이면 0 (비트 AND)	00000110 & 11111101	00000100
|	a | b	비트 별로 두 비트가 모두 0이면 0, 하나라도 1이면 1 (비트 OR)	00000110 | 00001000	00001110
^	a ^ b	비트 별로 두 비트가 서로 다르면 1, 같으면 0(비트 XOR)	00000110 ^ 00001111	00001001
~	~a	비트 별 반전 (0 → 1, 1 →0)	~00000110	11111001
>>	a >> b	a의 비트를 b 비트 수만큼 우측으로 이동 양수면 왼쪽 비트를 0으로 음수면 왼쪽 비트를 1로 채움	00010100 >> 2	00000101
<<	a << b	a의 비트를 b 비트 수만큼 좌측으로 이동 양수면 오른쪽 비트를 0으로 음수면 오른쪽 비트를 1로 채움	00010100 << 2	01010000
>>>	a >>> b	a의 비트를 b 비트 수만큼 우측으로 이동 부호 상관없이 왼쪽 비트를 0으로 채움	00010100 >>> 2	00000101

산술 연산자

종류	사용법	기능	사용 예	
			연산식	결과
=	a = b	b 값을 a에 대입	a = 2;	a = 2
+=	a += b	a값에 b를 더한 후, a에 대입 (a = a + b)	a +=2;	a = 4
-=	a-= b	a값에 b를 뺀 후, a에 대입 (a = a - b)	a -= 2;	a = 0
*=	a *= b	a값에 b를 곱한 후, (a = a * b)	a *= 2;	a = 4
/=	a /= b	a값에 b를 나눈 후, (a = a / b)	a /= 2;	a = 1
%=	a %= b	a값을 b로 나눈 나머지를 a에 대입 (a = a % b)	a %= 2;	a = 0

조건 연산자

종류	사용법	기능	사용 예	
			연산식	결과

?:	조건 ? 처리1: 처리2	조건이 참이면 처리 1 수행,
		거짓이면 처리 2 수행
			a = 2;
			b = 3;
			c = (a > b)? a : b ;
				c = 3

연산자 우선 순위

우선순위		연산자
1	함수 괄호 배열	( ) [ ] .
2	전치	++ -- ~ !
3	승제	* / %
4	가감	+ -
5	비트 이동	<< >> >>>
6	비교	< <= > >=
7	비교 (등식)	== !=
8	비트 AND	&
9	비트 XOR	^
10	비트 OR	|
11	논리 AND	&&
12	논리 OR	||
13	조건	?:
14	대입	= 및 축약 대입 연산자

8. 제어문

- 선택문 조건에 따라 선택적으로 문장 수행.
  - if문
    - ✓ if if(조건){ 실행문; }
    - ✓ if ~ else if(조건1){ 실행문; } else{ 실행문; }
    - ✓ if ~ else if ~ else if(조건1){ 실행문; } else if(조건2){ 실행문; } else{ 실행문; }
  - switch ~ case 문

```
swich( 조건 ){
        case 상수1 : 실행문; [break;]
        case 상수2 : 실행문; [break;]
        [default : 실행문;]
}
```

- 반복문 조건이 참인 동안에 문장이나 블록을 반복
  - for문

```
for( int i=0; (초기식) i>10; (조건식) i++(증감식) {  실행문;  }
```

  - while문

```
while(조건식){
        실행문;
}
```

  - do ~ while문 무조건 한 번은 실행

```
do{ 실행문; } while( 조건식 );
```

- 분기문 프로그램의 수행 흐름을 강제로 특정 위치로 이동
  - break문
    가장 가까운 반복 블록을 빠져 나올 때 사용
  - continue문
    이후의 문장은 실행하지 않고 그 다음 반복을 계속할 때 사용

9. 객체

- 객체 지향의 발전
  - ✓ 시뮬라 (1960) : 실용적 언어로 발전하지는 못하였으나 객체 지향 이론을 소개
  - ✓ 스몰톡, 이펠 (1970) : 순수 객체 지향언어로 인정받음. 객체지향 개념은 70,80년대를 거치면서 프로그램 언어뿐만 아니라 프로그램 개발 방법론 분야 등 많은 분야에서 발전. 그러나 범용적인 언어로 활용되지 못하였다.
  - ✓ C++ (1990) : C언어에 객체지향 개념을 도입한 C++이 객체 지향 언어로는 널리 이용되고 있다. 90년대에 들어 소프트웨어 품질 향상과 관련하여 대두된 객체지향 기술은 소프트웨어 부품화, 소프트웨어 컴포넌트의 재사용을 주요 목적으로 함. C++을 기반으로 자바가 등장함.
- 절차지향 언어와 객체지향 언어
  - ✓ 절차지향 언어는 공유된 데이터와 문제 해결 절차를 중시, 객체지향 언어는 프로그램을 객체들의 집합으로 보며 이러한 객체들이 프로그램의 중심이 된다.
  - ✓ 객체지향 언어는 절차지향 프로그래밍의 한계성을 극복

    구조체에서는 속성인 멤버 변수만 지정, 행위에 대해서는 함수로 구현하여 프로그램의 길이가 길어지고 재사용성이 거의 없으며 코드가 간결하지 못하다.
- 객체

  사람들이 의미를 부여하고 분류하는 논리적 단위, 실세계에 존재하는 명사로서의 성질을 갖춘 모든 것

  실제 프로그램에서 필요한 특성을 중심으로 모델링

속성 (정적인 특성)	**메소드 (동적인 특성)**
가격, 색상 . . .	달리다 , 먹다, 정지하다 . . .

- 클래스

  객체를 프로그래밍 언어로 정의한 것

  동일한 속성과 메서드를 가진 여러 개의 인스턴스 객체를 생성하는 형판

10. 객체지향 프로그래밍

- 1단계 - 실세계 객체 모델링

  실세계의 존재하는 객체로부터 프로그램에 필요한 특성을 모델링한다.

  ex) 자동차의 속성 (소유자, 색상, 제조회사), 메서드 (달리다, 정지하다)

- 2단계 - 클래스 정의

  ex) class car{

  String 모델명; String 제조사; void speedup(); void stop();

  }

- 3단계 - 인스턴스 객체의 생성 및 사용

  2단계에서 정의한 클래스로부터 실제로 존재하는 객체를 메모리에 생성, 사용

  ex) car morning = new Car("모닝", "현대");

  morning.speedup(); morning.stop();

11. 객체지향 언어의 특징

- 캡슐화 (자료 추상화)

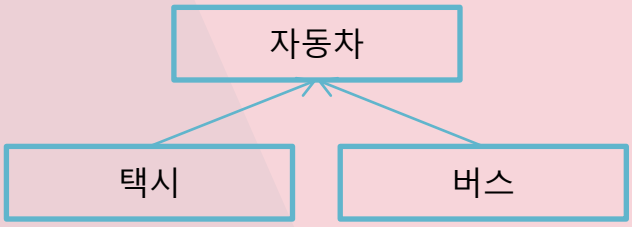
객체는 속성과 속성을 처리하는 메소드가 캡슐화되어 있어서 **객체를 사용하는 쪽에서는 캡슐화된 속성과 메소드가 실제 어떻게 처리되는지는 알 필요없이 단지 객체를 사용할 수 있는 인터페이스를 통해 사용만 하면 된다.** 객체 내부에서만 알고 있어야 하는 속성과 메소드는 외부에서 접근할 수 없게 해야 한다. 이를 위해 객체를 구성하는 속성과 메소드는 접근 제어자를 사용하여 접근 권한을 분명히 표시하고 있으며 이를 통해 캡슐화와 정보 은닉이 가능해진다.

➔ 객체 내부에서만 알고 있어야 하는 정보는 숨기고 외부에 보여야 할 정보만을 표현함으로써 프로그램을 간단하게 만드는 것. (서비스 코드 간결)

- 상속

기존의 클래스에서 모든 속성과 메소드를 상속받고 더 필요한 속성과 메소드를 추가하여 새로운 클래스를 생성하는 것

➔ 새로운 클래스가 기존의 클래스의 자료와 연산을 이용할 수 있게 하는 기능



택시 is a car
버스 is a car

- 다형성

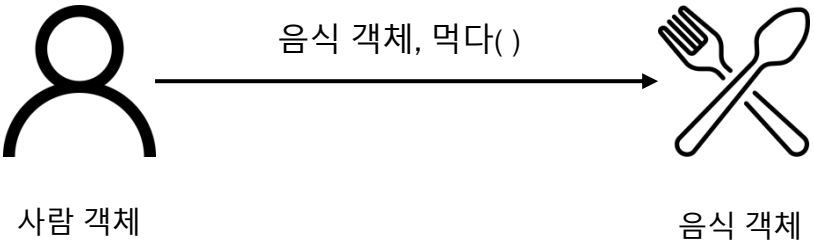
  여러 개의 형태를 갖는다는 의미로 메서드 형태가 다양하거나, 동일한 이름의 메서드가 약간씩 다른 의미를 가질 수 있다는 것.

  메서드 오버로딩 → 하나의 클래스에서 동일한 이름의 메소드가 매개 변수의 개수나 타입을 달리해서 여러 가지 형태로 존재하는 것

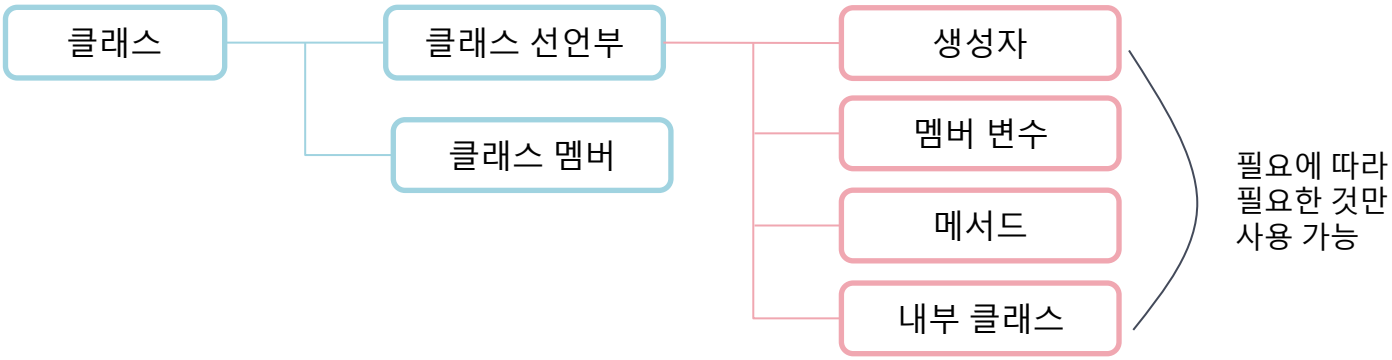
  메서드 오버라이딩 → 상속 관계에 있는 하위 클래스가 상위 클래스의 메서드를 재정의 하는 것

- 메시지

  인스턴스 객체 간에 서로 통신하는 방법으로 인스턴스 객체들은 메시지를 주고받으면서 일을 수행.



12. 클래스



+ 자바의 최상위 클래스 Object

인스턴스를 만들어서 사용할 경우 선언하지 않은 메서드나 속성이 코드 힌트로 뜬다?

1. Object는 자바의 최상위 클래스로서 사용자 클래스는 모두 Object를 상속한다.
2. 몇 가지 Object의 메서드는 클래스 생성시 오버라이딩 해서 사용한다.

   equals( )메서드 : 내용 비교

   toString( )메서드 : 객체의 클래스 이름과 메모리 참조 내용을 문자열로 리턴하며, print 메서드에 참조형 변수이름을 사용하면 자동 호출

   colone( )메서드 : 객체의 복제에 이용

객체 생성

- 1단계 : 객체 참조형 변수 선언

```
Student kim;
```

- 2단계 : 인스턴스 객체 생성

```
kim = new Student( );
```

- 1, 2단계 같이

```
Student kim = new Student( );
```

** new 연산자는 메모리를 할당하는 연산자 **

멤버 변수

클래스를 정의 할 때 메서드 밖에 선언하는 변수

객체가 가질 수 있는 정적인 속성을 나타냄

멤버변수의 접근제어자	사용범위				설명
	클래스	하위클래스	동일패키지	모든클래스	
private	O	X	X	X	자신의 클래스 안에서만 사용 가능 -> 캡슐화
default	O	X	O	X	동일 패키지내의 클래스에서만 접근가능
protected	O	O	O	X	동일 또는 하위 클래스에서만 접근가능
public	O	O	O	O	모든 클래스에서 접근 가능

접근제어의 필요성

- C언어로 된 프로그램에서 전역변수가 변경된 경우 → 전역변수가 사용된 나머지 모든 프로그램이 변경되어야 한다. → 일관된 수정이 어렵고 오류를 범하기 쉽다.
- 자바와 같은 객체지향 언어의 경우, 캡슐화 개념에 기초하여 객체의 상태를 나타내는 데이터는 그 객체의 메서드에 의해서만 변경되어야 한다. 다른 객체가 직접 접근하여 상태를 바꾸는 것은 불가능하거나 어렵게 하여야 한다.

final : 상수 final 메서드 : 재정의 불가능한 메서드 final 클래스 : 상속이 불가능한 클래스

객체 생성 static

✓ 전역변수(global object) 의 개념

✓ 클래스가 로딩되는 과정에서 기억공간이 한번만 확보

✓ 해당 클래스의 모든 객체가 변수를 공유한다.

✓ 객체들 사이의 통신이나 공통되는 속성을 표현하는데 사용한다.

✓ 일반 변수가 객체이름으로 접근하는데 반해 클래스 변수는 클래스 이름으로 접근한다.

변수	생성시기	생성 위치	생성 개수	사용 방법
클래스 변수	클래스 로딩 시 (객체 생성 이전)	클래스	1개	클래스 이름.클래스 변수이름 객체 이름.클래스 변수 이름
객체 변수	객체 생성 시	각 객체	객체 생성 수	객체이름.객체변수이름

13. 생성자

✓ 클래스로부터 객체를 생성할 때 객체의 초기화 과정을 담당하는 특수한 메소드로 객체가 생성될 때 무조건 수행된다.

✓ 특징

1. new 연산자와 함께 사용되며 객체 생성 시에 호출된다.
2. 객체 생성 시에 객체의 멤버 변수를 초기화하는 일을 주로 한다.
3. 생성자의 이름은 클래스 이름과 동일, 일반 적으로 첫 문자는 대문자이다.
4. return 유형이 없다.
5. 생성자가 없는 클래스는 자바 가상 머신이 자동으로 삽입하는 기본 생성자를 가짐.
6. 사용자가 정의한 생성자가 있을 경우 자바 가상 머신이 삽입한 기본 생성자는 자동으로 없어짐.

✓ 형식

[접근제어자] 생성자 이름 (매개변수1, 매개변수2,……)

✓ 생성자 메서드가 없는 클래스 : 생성자는 객체 생성시 반드시 호출되어야 하는 메서드. 그러나 반드시 생성자 메서드를 갖고 있을 필요는 없다. jvm이 자동으로 생성자를 삽입해서 생성.

✓ 생성자 오버로딩

매개변수의 개수, 타입, 순서를 달리하는 같은 이름의 생성자를 여러 개 정의할 수 있으며 이를 생성자 오버로딩 (overloading)이라고 한다. -> 다형성 구현

✓ this

현재 생성되어 사용 중인 객체의 자신을 의미.

this

객체 자신을 의미. this.속성 this.메서드 형태로 사용. 매개변수 이름이 멤버 변수 이름과 같을 때 많이 사용.

this( )

객체의 생성자 의미. 생성자 오버로딩에서 생상자 안에서 다른 생성자를 호출하는 경우 많이 사용.

14. 메소드와 오버로딩

- 메소드 선언

  메소드는 클래스가 가지는 동적인 특성인 행위를 정의.

  [접근 제어자] 리턴타입 메소드이름([매개 변수1], [매개 변수2]){ // 내용 }

  ex) public void test(int a, int b) { a = b; }

메소드에서 사용되는 접근 제어자

구분	접근 제어자	설명
접근 권한	public	모든 클래스에서 접근 가능
	default	동일 패키지에서만 접근 가능
	protected	동일, 하위 클래스 관계에서 접근 가능
	private	자신 클래스 안에서만 접근 가능
활용 방법	final	오버라이딩이 불가능한 메소드 정의
	static	클래스 메소드
	abstract	추상 메소드로, 하위 클래스에 의해 구현
	synchronized	스레드 동기화를 위한 메소드

- 클래스 메소드

  멤버 변수에 static으로 선언되는 클래스 변수가 있는 것처럼 메서드에도 static으로 선언되는 클레스 메서드가 있음.

  - 클래스를 로딩할 때 생김
  - 클래스로부터 생성된 모든 객체가 공유
  - 클래스 이름을 통해 접근
  - 일반 객체 변수 사용X, 클래수 변수만 사용

- 메소드 오버로딩
  - 생성자 오버로딩과 동일한 개념같은 클래스 안에 매개변수의 개수, 타입, 순서를 달리 하는 동일한 이름의 메서드가 여러 개 존재
  - 다형성 구현 X, 리턴타입과 접근제어자가 다른 것은 상관 없음

++ 왜 main( )메서드에 static을 붙일까? ++

일상적으로 static이 붙지 않는 메서드의 경우 메서드의 호출이 실행될 때 메서드를 할당 받는다. 그리고 실행이 종료 되면 메모리를 반환한다. 그에 반에 static이 붙은 qustnks 메서드는 호출 전에 메모리를 먼저 할당 받고 프로그램이 실행되는 동안 계속 상주한다. 따라서 main( )메서드는 자바 프로그램이 실행되는 동안, 호출하지 않더라도 메모리를 할당받고 항상 상주하고 있어야 하므로 static을 붙인다.

15. 상속

기존에 있는 클래스의 멤버 변수나 메소드를 물려받아 새로운 클래스를 만드는 것

(코드의 재사용성)

* JAVA는 단일상속을 지원 : 상위클래스는 하나만 지정한다. *

ex) 클래스 정의의 예

클래스명	자동차	택시	트럭
멤버변수	차기종, 색상, 속도, 제조회사	미터기	무게
메소드	속도를 올리다, 속도를 내리다 출발하다, 정지하다	미터기를 올리다 미터기를 내리다	짐을 싣다 짐을 내리다

- 상속 방법

[접근제어자] class 클래스 이름 extends 상위 클래스명

ex) public class Bus extends Car{ }

- 상속과 접근 제어자

protected인 경우 하위 클래스에서 접근할 수 있지만 private인 경우 상속 관계에서도 접근이 불가능하다.

** 상속과 생성자 **

메소드 호출의 기본 원칙 : 한 클래스에서 한 메소드가 호출되면

1. 클래스 내에 정의된 메소드인지 확인(있으면 클래스 내의 것을 실행)
2. 아니라면 상위 메소드에 정의된 메소드인지 확인
3. 최상위에 도달할 때까지 계속된다.

상위 클래스의 생성자메서드는 하위 클래스에 자동으로 상속되지 않는다.

어떤 클래스에 생성자메서드를 정의하지 않는다면 컴파일러는 기본 생성자메서드 메소드를 추가.

** 상속 시 유의 점 **

- 하위 멤버 변수를 재정의 시 상위 멤버변수를 덮어씀.
- 상속 시 private X
- 메소드는 자동상속
- 생성자 메서드 자동상속 X -> 상위클래스 super() 이용
- 상속관계의 반드시 매개 변수 없는 생성자 삽입

** super **

상위 클래스의 변수나 메소드를 참조하기 위해 사용하는 예약어

super()메서드 : 상위클래스의 생성자 메서드를 명시적으로 호출할 때 사용하는 것으로 하위 클래스의 생성자 메서드에서 제일 먼저 호출.

16. 추상클래스

상속 관계에 있는 클래스 중에서 상위 클래스에서는 특별한 구현 없이 사용하고자 하는 메서드만 기술하고 구체적인 구현은 하위 클래스에서 하도록 할 수 있음.

* 구체적인 내용을 기술 없이 모양만 갖춘 클래스

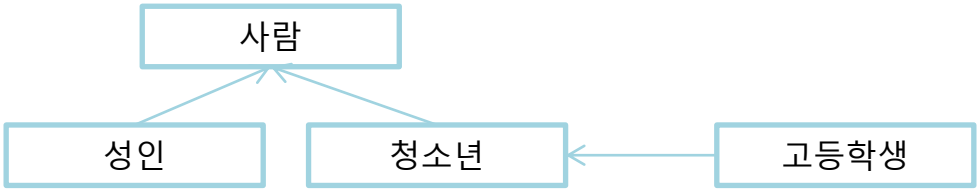
* 추상 클래스는 추상메서드를 가진다.

추상 클래스는 new연산자로 객체를 생성할 수 없다. 따라서 반드시 하위 클래스에서 상속받아 구현하여 사용하여야 한다.

형식

abstract class 클래스명{ //추상 메서드 선언 }

추상 클래스의 추상 메서드는 반드시 오버라이딩되어야 하기 때문에 하위 클래스들이 특정 메서드를 반드시 구현하도록 강제할 수 있고, 만약 추상 메서드를 오버라이딩하지 않으면 상속받는 클래스는 자동으로 추상 클래스가 된다.



17. 인터페이스

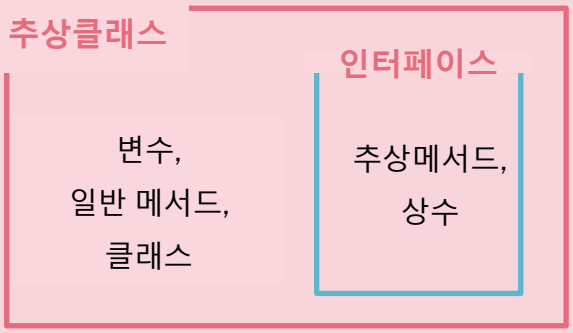
추상 메서드들의 모임

현재의 클래스가 이미 다른 클래스로부터 상속을 받고 있는 상태이면서, 또 다른 클래스의 요소들이 필요할 때 사용 (다중 상속의 유사한 기능을 흉내 낼 수 있음.

has a 관계 ex) 학생 has a 급식, 공부

형식

public interface 인터페이스명 { public 메소드 이름(인자들); }

구현

class 클래스명 implements 인터페이스명 { }